Qualcomm

"Exclusive" resources that are not exclusive - the case of shared GPIOs in DT systems

Bartosz Golaszewski
Senior Staff Engineer, Qualcomm



Tokyo, Linux Plumbers Conference 2025

Snapdragon and Qualcomm branded products are products of Qualcomm Technologies, Inc. and/or its subsidiaries.


Reality is often disappointing

Disconnect between device-tree and C implementation



Consider the following

```
&swr3 {
        status = "okay";

        pinctrl-0 = <&wsa2_swr_active>, <&spkr_23_sd_n_active>;
        pinctrl-names = "default";

        /* WSA8845, Right Woofer */
        right_woofer: speaker@0,0 {
                compatible = "sdw20217020400";
                reg = <0 0>;
                reset-gpios = <&lpass_tlmm 13 GPIO_ACTIVE_LOW>;
                #sound-dai-cells = <0>;
                sound-name-prefix = "WooferRight";
                vdd-1p8-supply = <&vreg_l15b_1p8>;
                vdd-io-supply = <&vreg_l12b_1p2>;
                qcom,port-mapping = <1 2 3 7 10 13>;
        };

        /* WSA8845, Right Tweeter */
        right_tweeter: speaker@0,1 {
                compatible = "sdw20217020400";
                reg = <0 1>;
                reset-gpios = <&lpass_tlmm 13 GPIO_ACTIVE_LOW>;
                #sound-dai-cells = <0>;
                sound-name-prefix = "TweeterRight";
                vdd-1p8-supply = <&vreg_l15b_1p8>;
                vdd-io-supply = <&vreg_l12b_1p2>;
                qcom,port-mapping = <4 5 6 7 11 13>;
        };
};
```

Consider the following

```
&swr3 {
        status = "okay";

        pinctrl-0 = <&wsa2_swr_active>, <&spkr_23_sd_n_active>;
        pinctrl-names = "default";

        /* WSA8845, Right Woofer */
        right_woofer: speaker@0,0 {
                compatible = "sdw20217020400";
                reg = <0 0>;
                reset-gpios = <&lpass_tlmm 13 GPIO_ACTIVE_LOW>;
                #sound-dai-cells = <0>;
                sound-name-prefix = "WooferRight";
                vdd-1p8-supply = <&vreg_l15b_1p8>;
                vdd-io-supply = <&vreg_l12b_1p2>;
                qcom,port-mapping = <1 2 3 7 10 13>;
        };

        /* WSA8845, Right Tweeter */
        right_tweeter: speaker@0,1 {
                compatible = "sdw20217020400";
                reg = <0 1>;
                reset-gpios = <&lpass_tlmm 13 GPIO_ACTIVE_LOW>;
                #sound-dai-cells = <0>;
                sound-name-prefix = "TweeterRight";
                vdd-1p8-supply = <&vreg_l15b_1p8>;
                vdd-io-supply = <&vreg_l12b_1p2>;
                qcom,port-mapping = <4 5 6 7 11 13>;
        };
};
```

Now consider this

```
static int gpiod_request_commit(struct gpio_desc *desc, const char *label)
{
        unsigned int offset;
        int ret;

        CLASS(gpio_chip_guard, guard)(desc);
        if (!guard.gc)
                return -ENODEV;

        if (test_and_set_bit(GPIOD_FLAG_REQUESTED, &desc->flags))
                return -EBUSY;
```

Now consider this

```
static int gpiod_request_commit(struct gpio_desc *desc, const char *label)
{
        unsigned int offset;
        int ret;

        CLASS(gpio_chip_guard, guard)(desc);
        if (!guard.gc)
                return -ENODEV;

        if (test_and_set_bit(GPIOD_FLAG_REQUESTED, &desc->flags))
                return -EBUSY;
```

How about just not caring? ¯_(ツ)_/¯

```
if (!(ret == -EBUSY && flags & GPIOD_FLAGS_BIT_NONEXCLUSIVE))
    return ERR_PTR(ret);

/*
* This happens when there are several consumers for
* the same GPIO line: we just return here without
* further initialization. It is a bit of a hack.
* This is necessary to support fixed regulators.
*
* FIXME: Make this more sane and safe.
*/
dev_info(consumer, "nonexclusive access to GPIO for %s\n", name);
return desc;
```

How about just not caring? ¯_(ツ)_/¯

```
if (!(ret == -EBUSY && flags & GPIOD_FLAGS_BIT_NONEXCLUSIVE))
    return ERR_PTR(ret);

/*
* This happens when there are several consumers for
* the same GPIO line: we just return here without
* further initialization. It is a bit of a hack.
* This is necessary to support fixed regulators.
*
* FIXME: Make this more sane and safe.
*/
dev_info(consumer, "nonexclusive access to GPIO for %s\n", name);
return desc;
```

GPIOD_FLAGS_BIT_NONEXCLUSIVE



GPIOD_FLAGS_BIT_NONEXCLUSIVE

- Just gives an already requested GPIO descriptor to every interested driver and lets users “fight it out”
- No synchronization
- No enable-count
- No configuration verification
- Driver needs to know that it shared the GPIO (it has no business knowing it)

How about just not caring? ¯_(ツ)_/¯

```
if (!(ret == -EBUSY && flags & GPIOD_FLAGS_BIT_NONEXCLUSIVE))
    return ERR_PTR(ret);

/*
* This happens when there are several consumers for
* the same GPIO line: we just return here without
* further initialization. It is a bit of a hack.
* This is necessary to support fixed regulators.
*
* FIXME: Make this more sane and safe.
*/
dev_info(consumer, "nonexclusive access to GPIO for %s\n", name);
return desc;
```

How about just not caring? ¯_(ツ)_/¯

```
if (!(ret == -E                                    XCLUSIVE))
    return ERR_

/*
* This happens
* the same GPIO
* further initi
* This is neces
*
* FIXME: Make t
*/
dev_info(consum                                     %s\n", name);
return desc;
```



reset-gpio



reset-gpio

```
&swr3 {
        status = "okay";

        pinctrl-0 = <&wsa2_swr_active>, <&spkr_23_sd_n_active>;
        pinctrl-names = "default";

        /* WSA8845, Right Woofer */
        right_woofer: speaker@0,0 {
                compatible = "sdw20217020400";
                reg = <0 0>;
                reset-gpios = <&lpass_tlmm 13 GPIO_ACTIVE_LOW>;
                #sound-dai-cells = <0>;
                sound-name-prefix = "WooferRight";
                vdd-1p8-supply = <&vreg_l15b_1p8>;
                vdd-io-supply = <&vreg_l12b_1p2>;
                qcom,port-mapping = <1 2 3 7 10 13>;
        };

        /* WSA8845, Right Tweeter */
        right_tweeter: speaker@0,1 {
                compatible = "sdw20217020400";
                reg = <0 1>;
                reset-gpios = <&lpass_tlmm 13 GPIO_ACTIVE_LOW>;
                #sound-dai-cells = <0>;
                sound-name-prefix = "TweeterRight";
                vdd-1p8-supply = <&vreg_l15b_1p8>;
                vdd-io-supply = <&vreg_l12b_1p2>;
                qcom,port-mapping = <4 5 6 7 11 13>;
        };
};
```

reset-gpio

- The reset subsystem already allows sharing reset control handles (with `reset_control_get_shared()`
- It counts reset assertions
- reset-gpios use-case is simple: an output pin that we drive high or low

Question

Can we create an arbitrary device not represented in device-tree?

Answer

Question

Can we create an arbitrary device not represented in device-tree?

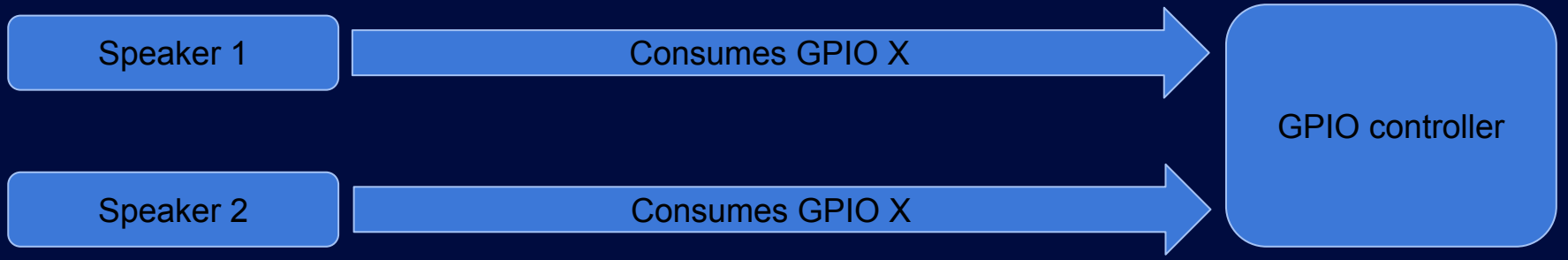
Answer

Of course! This is what the auxiliary bus is for!

reset-gpio

- Idea: create an auxiliary reset controller device between the GPIO consumer and supplier

reset-gpio


Speaker 1
Consumes GPIO X
Speaker 2
Consumes GPIO X
GPIO controller

reset-gpio


Speaker 1
Speaker 2
Consumes reset
Consumes reset
reset-gpio
Consumes GPIO X
GPIO controller

reset-gpio

- The device is created ad-hoc when the first device with a “reset-gpios” property requests a reset control
- Upstream status:
  - Available and used by Qualcomm platforms
  - Still in the process of being reworked/improved

GPIOLIB-managed shared GPIOs



Shared GPIOs

```
&swr3 {
        status = "okay";

        pinctrl-0 = <&wsa2_swr_active>, <&spkr_23_sd_n_active>;
        pinctrl-names = "default";

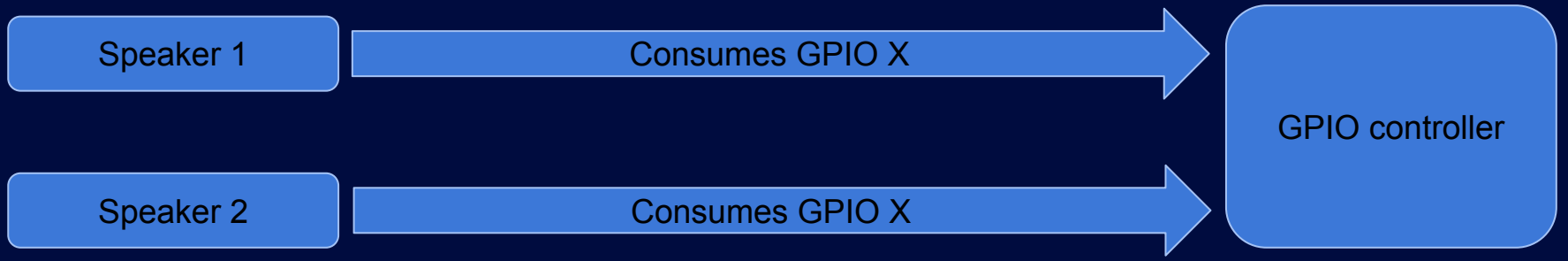
        /* WSA8845, Right Woofer */
        right_woofer: speaker@0,0 {
                compatible = "sdw20217020400";
                reg = <0 0>;
                powerdown-gpios = <&lpass_tlmm 13 GPIO_ACTIVE_LOW>;
                #sound-dai-cells = <0>;
                sound-name-prefix = "WooferRight";
                vdd-1p8-supply = <&vreg_l15b_1p8>;
                vdd-io-supply = <&vreg_l12b_1p2>;
                qcom,port-mapping = <1 2 3 7 10 13>;
        };

        /* WSA8845, Right Tweeter */
        right_tweeter: speaker@0,1 {
                compatible = "sdw20217020400";
                reg = <0 1>;
                powerdown-gpios = <&lpass_tlmm 13 GPIO_ACTIVE_LOW>;
                #sound-dai-cells = <0>;
                sound-name-prefix = "TweeterRight";
                vdd-1p8-supply = <&vreg_l15b_1p8>;
                vdd-io-supply = <&vreg_l12b_1p2>;
                qcom,port-mapping = <4 5 6 7 11 13>;
        };
};
```

Shared GPIOs

- Shared GPIOs with functions other than “reset” cannot use the reset-gpio driver
- Problems with sharing GPIOs:
  - GPIOs have direction
  - GPIOs have configuration settings
    - pull-up/down
    - open drain/source
    - active-low
  - GPIOs have binary state (high/low)
  - GPIOs raise interrupts
- Sharing GPIOs is not as straightforward as in most other subsystems
  - It’s why they were meant to be an exclusive resource going through some higher-level abtractions

Shared GPIOs



Shared GPIOs



Shared GPIOs

1. At init: scan device-tree and look for GPIOs referenced more than once by phandle
2. When a GPIO controller exposing shared GPIOs is bound: create proxy GPIO device that manages access to the shared GPIO in that controller for every user
3. When the user requests a shared GPIO: create a fallback machine lookup entry pointing to the GPIO exposed by the proxy device and let the user request it, instead of the real, shared GPIO

gpio-shared-proxy logic

- Shared GPIO has a use count and an enable count
- The first user to request a GPIO will set its direction, the subsequent user must use the same direction or will see an error

gpio-shared-proxy logic

- In input mode:
  - Users can freely read the value and request interrupts
- In output mode:
  - An “enabled” shared GPIO is driven high
  - Users “vote” to enable a shared GPIO when they call `gpiod_set_value(d, 1)`
    - GPIO is driven high when at least one user votes to drive it high
    - GPIO is driven low when all users vote to drive it low

gpio-shared-proxy logic

- Config options:
  - Shared options:
    - There’s no good solution
    - Allow drivers to change config but warn if settings conflict
  - Per-user options:
    - GPIO name, active-low can be freely changed

Shared GPIOs

- Status: queued for v6.19

Interaction between GPIO and pinctrl



GPIO vs pinctrl

```
vreg_edp_3p3: regulator-edp-3p3 {
    compatible = "regulator-fixed";

    regulator-name = "VCC3LCD";
    regulator-min-microvolt = <3300000>;
    regulator-max-microvolt = <3300000>;

    gpio = <&tlmm 25 GPIO_ACTIVE_HIGH>;
    enable-active-high;

    pinctrl-names = "default";
    pinctrl-0 = <&edp_reg_en>;

    regulator-boot-on;
};
```

```
&tlmm {
        edp_reg_en: edp-reg-en-state {
                pins = "gpio25";
                function = "gpio";
                drive-strength = <16>;
                bias-disable;
        };
};
```

GPIO vs pinctrl

```
vreg_edp_3p3: regulator-edp-3p3 {
    compatible = "regulator-fixed";

    regulator-name = "VCC3LCD";
    regulator-min-microvolt = <3300000>;
    regulator-max-microvolt = <3300000>;

    gpio = <&tlmm 25 GPIO_ACTIVE_HIGH>;
    enable-active-high;

    pinctrl-names = "default";
    pinctrl-0 = <&edp_reg_en>;

    regulator-boot-on;
};
```

```
&tlmm {
        edp_reg_en: edp-reg-en-state {
                pins = "gpio25";
                function = "gpio";
                drive-strength = <16>;
                bias-disable;
        };
};
```

GPIO vs pinctrl

- Device-tree feeds information to different subsystems
- They can conflict in weird ways unknowingly to the user

GPIO vs pinctrl

- GPIOs are exclusive so if a GPIO is muxed by the pinctrl subsystem, it can no longer be requested by the GPIO subsystem and the non-exclusive flag doesn’t help

GPIO vs pinctrl

- GPIOs are exclusive so if a GPIO is muxed by the pinctrl subsystem, it can no longer be requested by the GPIO subsystem and the non-exclusive flag doesn’t help
- Workaround?
  - The .strict pinconf flag set to false

GPIO vs pinctrl

- Solution: new class of pin functions: GPIO functions
- `PINFUNCTION_FLAG_GPIO`
- Pins assigned a function that’s marked as a GPIO can be requested by GPIOLIB
- Pin control drivers need to describe functions as GPIOs



Thank you

Nothing in these materials is an offer to sell any of the components or devices referenced herein.

© Qualcomm Technologies, Inc. and/or its affiliated companies. All Rights Reserved.

Qualcomm and Snapdragon are trademarks or registered trademarks of Qualcomm Incorporated. Other products and brand names may be trademarks or registered trademarks of their respective owners.

References in this presentation to “Qualcomm” may mean Qualcomm Incorporated, Qualcomm Technologies, Inc., and/or other subsidiaries or business units within the Qualcomm corporate structure, as applicable. Qualcomm Incorporated includes our licensing business, QTL, and the vast majority of our patent portfolio. Qualcomm Technologies, Inc., a subsidiary of Qualcomm Incorporated, operates, along with its subsidiaries, substantially all of our engineering, research and development functions, and substantially all of our products and services businesses, including our QCT semiconductor business.

Snapdragon and Qualcomm branded products are products of Qualcomm Technologies, Inc. and/or its subsidiaries. Qualcomm patents are licensed by Qualcomm Incorporated.

Follow us on:

For more information, visit us at qualcomm.com & qualcomm.com/blog